

RIT*: Riemannian Informed Trees for Anisotropic Motion Planning

Author Names Omitted for Review Affiliation Omitted for Review

Abstract—Asymptotically-optimal sampling-based planners accelerate convergence by focusing sampling within an informed set, but all existing methods define this set using Euclidean distance. When traversal costs are anisotropic—as with joint-inertia weighting or obstacle-proximity penalties—the Euclidean informed set wastes samples in regions that cannot contain improving paths. We present RIT* (Riemannian Informed Trees), which replaces the Euclidean informed set, neighbourhood ball, and edge cost with their Riemannian counterparts, yielding a provably smaller sampling region. RIT* includes a Collision-Adaptive Riemannian Metric (CARM) module that learns a cost field online from collision feedback, requiring no a priori obstacle model. Experiments on 2-D through 6-DOF UR10e manipulator environments show that RIT* reduces path cost by up to 11% over five baselines in constrained manipulator planning, while CARM recovers up to 27% of the oracle-metric advantage using only a binary collision checker.

Index Terms—Motion planning, Riemannian geometry, asymptotic optimality, informed sampling, online metric learning.

I. INTRODUCTION

Manipulator motion planning requires optimizing paths under non-Euclidean costs: joint-inertia weighting penalizes motion of high-inertia joints, obstacle-proximity penalties steer paths away from collisions, and task-space norms encode end-effector preferences [14], [15]. A natural framework for such costs is Riemannian geometry, where a spatially-varying metric tensor $G(x)$ encodes directional traversal cost. Current asymptotically-optimal (AO) planners—from RRT* [2] through Informed RRT* [3], BIT* [4], AIT*/EIT* [5], to the recent APT* [6]—accelerate convergence by sampling within an *informed set*, but all define this set using Euclidean distance. When the true cost metric is anisotropic, the Euclidean informed set I_E is a strict superset of the cost-relevant region I_R : for a 6-DOF arm with joint-inertia condition number $\kappa = 12$, I_E is approximately $3.4\times$ larger than necessary.

Two problems remain open. First, no batch-informed planner uses Riemannian geometry for its global sampling strategy—informed set, neighbourhood, and edge cost—as opposed to local steering alone. Second, no sampling-based planner learns a suitable cost metric online from planning experience.

Contributions. We propose **RIT*** (Riemannian Informed Trees), which addresses both gaps:

- 1) **Online metric learning via CARM.** The Collision-Adaptive Riemannian Metric (CARM) module is, to our knowledge, the first online metric-learning scheme for sampling-based planning that requires no a priori obstacle model, no demonstrations, and no auxiliary sensors—only the binary collision checker already available to any planner. CARM places Gaussian kernels

on discovered collision points to build an adaptive cost field that inflates traversal cost near obstacles, and feeds the learned metric directly into the planning loop (section III-G).

- 2) **Riemannian informed planning pipeline.** RIT* replaces every Euclidean primitive in the batch-informed pipeline with its Riemannian counterpart: (i) a whitening transform maps the Riemannian informed set I_R to a standard prolate hyperspheroid for efficient uniform sampling; (ii) neighbours are found within ellipsoidal Riemannian balls that reach further along cheap directions; (iii) a three-level cascading edge evaluation scheme filters the majority of candidate edges before expensive integration. This framework makes CARM’s learned metric actionable throughout the planning loop.
- 3) **Experimental validation.** Multi-trial experiments across eight environments (2-D through 6-DOF UR10e manipulator) against five baselines show that RIT* achieves up to 11% cost reduction in 6-DOF manipulator planning with statistical significance. CARM recovers up to 27% of the oracle-metric advantage without any prior cost model.

Euclidean informed planners. Gammell et al. [3] introduced direct sampling of the prolate hyperspheroid, which BIT* [4] combines with batch sampling and lazy edge evaluation. AIT* and EIT* [5] add asymmetric bidirectional search. APT* [6] uses Coulomb-force-inspired virtual forces to prolate the neighbourhood ball and adapts batch size. FIT* [7] adjusts batch size for faster initial convergence. All define the informed set using Euclidean distance.

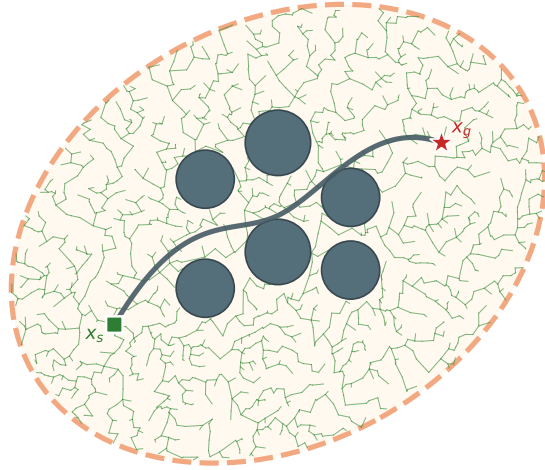
Riemannian methods in planning. CHOMP [14] and STOMP [15] optimise trajectories under Riemannian-like costs but are trajectory optimisers without AO guarantees. Kyaw and Kelly [8] propose a Riemannian RRT* using retractions for local steering; their planner is incremental (not batch-informed), does not construct an informed set, and does not learn metrics online. Bi-AM-RRT* [9] uses a diffusion-based metric to bias nearest-neighbour selection in an incremental framework. Kim et al. [13] use Riemannian metrics for steering on constrained manifolds. None integrate Riemannian geometry into the global informed sampling strategy of a batch-informed planner.

Obstacle-aware Riemannian metrics. Klein et al. [10] design region-avoiding metrics using hand-crafted barrier functions requiring full obstacle geometry. RMPflow [11] composes task-space metrics for reactive control. Beik-Mohammadi et al. [12] learn Riemannian manifolds from demonstrations for reactive motion generation. All require a priori obstacle geometry or demonstrations—CARM re-

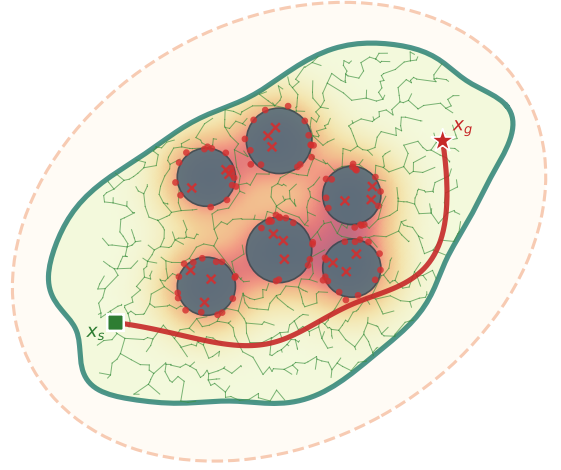
RIT*: Riemannian Informed Trees for Faster Motion Planning

(a) Euclidean Informed Set (I_E)

(b) Riemannian Informed Trees (RIT*)



I_E — Euclidean informed set



$I_R \subset I_E$

— I_E (Euclidean — too large) — I_R (Riemannian — tight) — Euclidean tree — CARM tree — Euclidean path — Our path (CARM)

Fig. 1: Overview of RIT*. (a) Euclidean planners sample uniformly from the informed set I_E (orange ellipse), wasting samples in high-cost regions near obstacles. (b) RIT* replaces every Euclidean primitive with its Riemannian counterpart: the tighter informed set $I_R \subset I_E$ (teal) eliminates wasted volume; the anisotropic neighbourhood ball reaches further along cheap directions; a three-level cascade rejects the majority of candidate edges before expensive integration; and the CARM module learns a cost field online from collision feedback, progressively tightening I_R .

quires neither.

II. PROBLEM FORMULATION

Configuration Space and Metric

Let $\mathcal{C} \subseteq \mathbb{R}^d$ be the *configuration space* of a robotic system with d degrees of freedom. Each point $x \in \mathcal{C}$ is a configuration vector (e.g., a vector of joint angles for a d -DOF manipulator). The space is equipped with a *Riemannian metric tensor field*

$$G : \mathcal{C} \rightarrow \mathbb{S}_{++}^d,$$

where $\mathbb{S}_{++}^d$ denotes the set of $d \times d$ symmetric positive-definite matrices. At every configuration x , the matrix $G(x)$ assigns a local inner product to pairs of velocity vectors: intuitively, directions along which the robot is heavy or inertially expensive to move are *stretched* by $G(x)$, so the planner naturally favours moving lighter joints.

Path Cost

A *path* is a piecewise-smooth curve $\sigma : [0, 1] \rightarrow \mathcal{C}$, where $\sigma(0)$ is the start and $\sigma(1)$ is the goal configuration. The

quantity $\dot{\sigma}(t) \in \mathbb{R}^d$ is the *tangent vector* (velocity) of the path at parameter value t . The *Riemannian arc-length* of σ is

$$c(\sigma) = \int_0^1 \underbrace{\sqrt{\underbrace{\dot{\sigma}(t)^\top}_{\text{row}} \underbrace{G(\sigma(t))}_{\substack{\text{local metric} \\ \text{matrix}}} \underbrace{\dot{\sigma}(t)}_{\text{col. velocity}}}}_{\text{instantaneous Riemannian speed}} dt. \quad (1)$$

When $G(x) = I_d$ (the identity matrix) for all x , eq. (1) reduces to the ordinary Euclidean arc-length $\int_0^1 \|\dot{\sigma}(t)\|_2 dt$. For a manipulator, setting $G(q) = M(q)$ (the joint-space inertia matrix) makes $c(\sigma)$ proportional to the total kinetic energy exerted along the path, penalising motion of high-inertia joints (shoulder, elbow) more than low-inertia joints (wrist).

Geodesic Distance and Optimal Path

The *geodesic distance* between two configurations $x, y \in \mathcal{C}$ is the infimum of $c(\sigma)$ over all paths connecting them:

$$d_R(x, y) = \inf_{\sigma : \sigma(0)=x, \sigma(1)=y} c(\sigma). \quad (2)$$

When $G = I_d$ this equals the Euclidean distance $\|x - y\|_2$.

Let $\mathcal{C}_{\text{free}} \subseteq \mathcal{C}$ be the *obstacle-free* subset of configurations (those for which the robot does not collide with any obstacle).

Given a *start* configuration $x_s \in \mathcal{C}_{\text{free}}$ and a *goal* configuration $x_g \in \mathcal{C}_{\text{free}}$, the optimal motion-planning problem is to find

$$\sigma^* = \arg \min_{\substack{\sigma : \sigma(0)=x_s, \sigma(1)=x_g \\ \sigma(t) \in \mathcal{C}_{\text{free}} \forall t \in [0,1]}} c(\sigma). \quad (3)$$

The cost of the optimal path is denoted c^* . A planner is called *asymptotically optimal* (AO) if the cost c_n of the best solution found after n samples satisfies $c_n \rightarrow c^*$ almost surely as $n \rightarrow \infty$.

Informed Sets

Once a feasible path of cost c_{best} has been found, any configuration that cannot lie on a strictly cheaper path can be safely ignored. Formally, only configurations x satisfying $d_R(x_s, x) + d_R(x, x_g) \leq c_{\text{best}}$ (i.e., the detour through x cannot beat the current best) need be sampled. This defines two nested sets.

The *Euclidean informed set*

$$I_E = \left\{ x \in \mathcal{C} \mid \underbrace{\|x_s - x\|_2}_{\text{straight-line dist. start} \rightarrow x} + \underbrace{\|x - x_g\|_2}_{\text{straight-line dist. } x \rightarrow \text{goal}} \leq c_{\text{best}} \right\} \quad (4)$$

is the standard prolate hyperspheroid used by Informed RRT* and BIT*. Because $\|\cdot\|_2$ lower-bounds the true geodesic distance, I_E is a superset of the true admissible region: it may contain many configurations that cannot actually improve the solution.

The *Riemannian informed set*

$$I_R = \left\{ x \in \mathcal{C} \mid \underbrace{d_R(x_s, x)}_{\text{Riemannian dist. start} \rightarrow x} + \underbrace{d_R(x, x_g)}_{\text{Riemannian dist. } x \rightarrow \text{goal}} \leq c_{\text{best}} \right\} \quad (5)$$

uses the true Riemannian geodesic distance, so it is a tighter bound. Whenever $G(x) \succeq I_d$ (i.e., $G(x) - I_d$ is positive semi-definite), the Cauchy–Schwarz inequality gives $d_R(x, y) \geq \|x - y\|_2$, which implies $I_R \subseteq I_E$. In anisotropic spaces (large condition number $\kappa(G)$), I_R can be dramatically smaller than I_E , concentrating samples where they matter and accelerating convergence.

III. METHOD

A. Overview

Our approach builds on BIT* (Batch Informed Trees) [4], a state-of-the-art asymptotically-optimal planner that combines batch sampling with lazy edge evaluation. BIT* operates in iterations: at each iteration it draws a batch of B samples from the *Euclidean informed set* I_E —the prolate hyperspheroid containing all configurations that could lie on a path shorter than the current best cost c_{best} . It then connects each sample to nearby vertices whose Euclidean distance falls within a shrinking connection radius $r_n = \gamma (\log n/n)^{1/d}$, evaluates candidate parent edges in order of estimated cost-to-come, rewires the tree when cheaper connections become available, and prunes vertices that can no longer contribute to improving paths. This informed-set-driven strategy focuses computational effort where improvements are most likely, making BIT* significantly faster than RRT* at converging to the optimal path.

However, every geometric primitive in BIT*—the informed set, the neighbourhood ball, and the edge cost—is defined using Euclidean distance, which is suboptimal when the true traversal cost is anisotropic or spatially varying.

RIT* (Riemannian Informed Trees) extends BIT* by replacing each of these Euclidean primitives with its Riemannian counterpart, yielding a planner that natively accounts for non-Euclidean cost structures. Concretely, RIT* introduces four modifications: (1) the informed set becomes the *Riemannian* informed set $I_R \subseteq I_E$, which is provably smaller and eliminates samples in high-cost regions (section III-C); (2) nearest-neighbour search uses ellipsoidal Riemannian balls that stretch along cheap directions and compress along expensive ones (section III-D); (3) edge costs are evaluated through a three-level cascading scheme that amortises the expense of numerical integration (section III-E); and (4) when no metric is known a priori, the Collision-Adaptive Riemannian Metric (CARM) module learns the cost landscape online from collision feedback (section III-G). Additionally, RIT* rewires the tree under Riemannian costs, ensuring that cheaper connections discovered during the search propagate correctly through the tree (section III-F). fig. 2 illustrates these six sub-steps within a single iteration.

B. Algorithm Description

Inputs. x_s, x_g are the start/goal configurations, $\mathcal{C}_{\text{free}} \subset \mathbb{R}^d$ is the free configuration space, $G(\cdot)$ is the base Riemannian metric tensor field, B is the per-iteration sample batch size, and T is the iteration budget. **Output.** The optimal Riemannian path $\sigma^* : [0, 1] \rightarrow \mathcal{C}_{\text{free}}$ from x_s to x_g (or $\emptyset$ if none is found within T iterations).

algorithm 1 formalises the complete procedure. The planner initialises the tree with the start vertex and precomputes a metric cache for $O(1)$ lookups (lines 1–2). Each iteration then has three compact phases. First, the *prune-and-sample phase* (lines 3–8) prunes vertices, refreshes the informed set (once a solution exists), draws a fresh batch, and computes the Riemannian radius. Second, the *edge-queue phase* (lines 9–17) filters admissible, collision-free samples, builds a BIT*-style queue ordered by $f(e) = g(v) + \hat{c}_R(v, x) + \hat{h}_R(x)$, applies the L1→L2→L3 cascade, updates/rewires parent links, and records collision points. Third, the *adaptive-metric phase* (lines 18–19) rebuilds the metric cache only when CARM is active, the rebuild interval is met, and new collision data has arrived. After the loop, the extracted path is shortcut-smoothed and its final cost is recomputed exactly (lines 20–21). The following subsections detail each component and reference the corresponding algorithm lines.

Having established the overall structure, we now detail each component, beginning with the sampling strategy that forms the foundation of informed planning.

C. Riemannian Informed Sampling

The quality of an informed planner depends critically on how tightly its sampling region approximates the set of configurations that can actually improve the current solution. In BIT*, this region is the Euclidean informed set I_E —a prolate

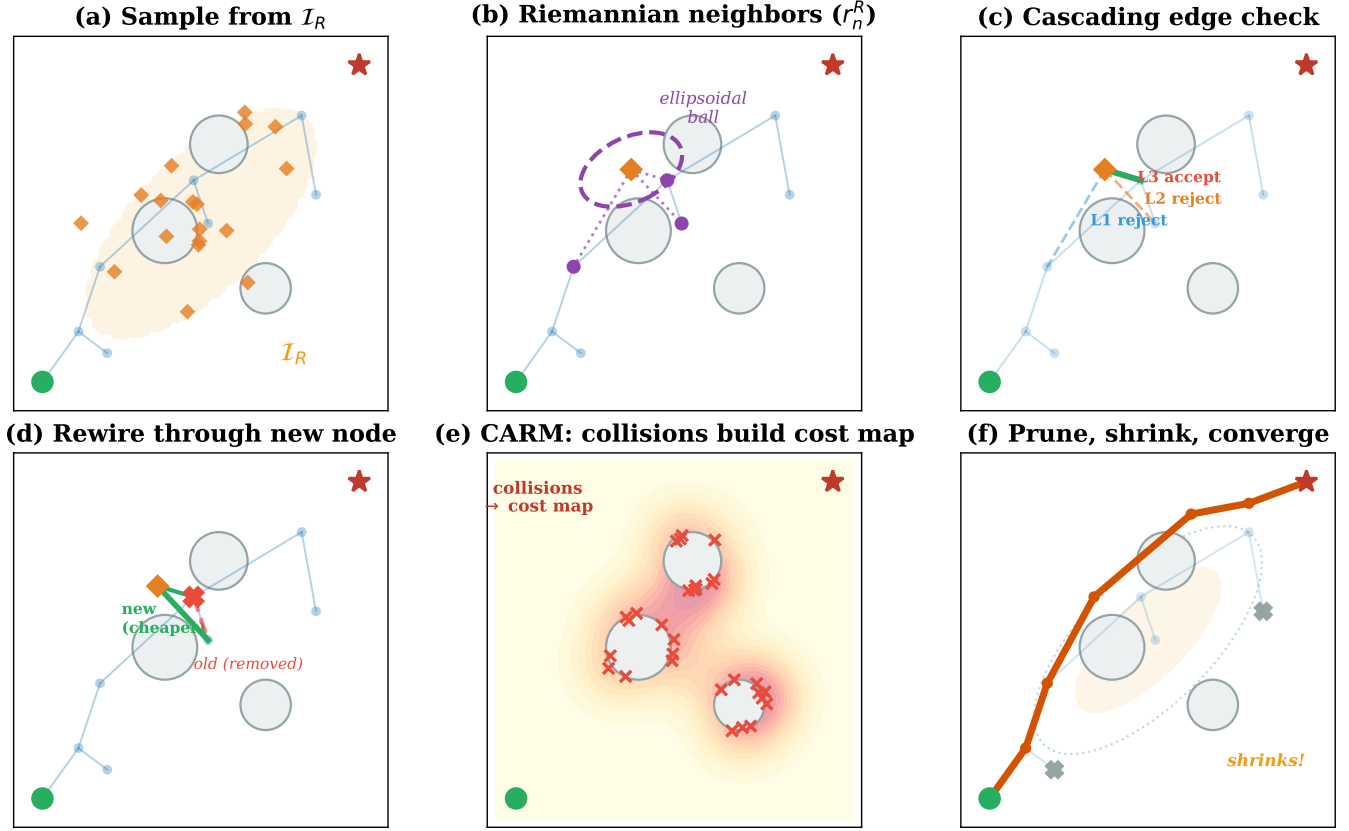


Fig. 2: Per-iteration pipeline of RIT*. (a) Samples drawn from the Riemannian informed set I_R . (b) Neighbours identified within ellipsoidal Riemannian radius r_n^R . (c) Three-level cascade filters candidate edges (L1→L2→L3). (d) Rewiring via cheaper Riemannian connections. (e) CARM collects collision points and rebuilds the metric field. (f) Pruning shrinks the informed set as c_{best} decreases.

hyperspheroid defined by $\|x_s - x\|_2 + \|x - x_g\|_2 \leq c_{\text{best}}$. When the cost metric is anisotropic, I_E is unnecessarily large: it includes configurations whose Riemannian cost-to-come plus cost-to-go already exceeds c_{best} .

RIT* replaces I_E with the Riemannian informed set I_R (line 8 of algorithm 1), defined by (5) using geodesic distances. Since $d_R(x, y) \geq \|x - y\|_2$ whenever $G \succeq I_d$, we have $I_R \subseteq I_E$ —every sample drawn from I_R is guaranteed to lie in a region where path improvement is possible under the true cost, whereas many samples from I_E would fall in regions where the Riemannian cost is too high.

To sample uniformly from I_R , we apply a whitening transform. We compute the average metric tensor $\bar{G} = \mathbb{E}[G(x)]$ along the line from x_s to x_g and form its Cholesky factor $L = \text{chol}(\bar{G})$. The coordinate change $\tilde{x} = Lx$ maps I_R to an approximate prolate hyperspheroid in the whitened space, from which uniform sampling is straightforward using the standard technique of [3]. Samples are then mapped back to the original space via $x = L^{-1}\tilde{x}$ and clipped to configuration-space bounds. This procedure yields uniform coverage of I_R in the Riemannian volume measure.

The volume reduction achieved by sampling from I_R instead

of I_E is characterised by the ratio

$$\frac{\text{Vol}(I_R)}{\text{Vol}(I_E)} = \prod_{i=1}^d \sqrt{\frac{\lambda_{\min}}{\lambda_i}}, \quad (6)$$

where $\lambda_1 \leq \dots \leq \lambda_d$ are the eigenvalues of G . For a 6-DOF manipulator with joint-inertia condition number $\kappa = 12$, the Riemannian set has approximately 0.29 times the Euclidean volume, eliminating 71% of wasted samples. fig. 3 illustrates the difference: I_E covers a large region including high-cost zones near obstacles, while I_R focuses samples on the subspace where cost improvement is geometrically possible.

With samples now focused on the most promising region, the next step is to identify which existing tree vertices each new sample should connect to.

D. Riemannian Nearest-Neighbour Search

After drawing samples from I_R , the planner must identify candidate parent vertices for each new sample (line 9 of algorithm 1). In BIT*, neighbours are found within a Euclidean ball of radius r_n —an isotropic sphere that treats all directions equally. This is a mismatch when the cost metric is anisotropic: the planner may connect to vertices that are close in Euclidean distance but expensive in Riemannian cost, while missing

Algorithm 1: RIT* planner

Input: Start x_s , goal x_g , free space $\mathcal{C}_{\text{free}}$, metric $G(\cdot)$, batch size B , iterations T , CARM update interval Δ , CARM parameters σ, α

Output: Optimal path σ^*

```

1  $\mathcal{V} \leftarrow \{x_s\}; \mathcal{E} \leftarrow \emptyset; c_{\text{best}} \leftarrow \infty; \mathcal{C}_{\text{col}} \leftarrow \emptyset;$ 
2 Precompute metric cache  $\mathcal{M}$ ;
3 for  $t = 1, \dots, T$  do
4    $X_{\text{new}} \leftarrow \text{RIEMANNIANINFORMEDSAMPLE}(B, c_{\text{best}}, G);$ 
5    $\mathcal{V} \leftarrow \mathcal{V} \cup X_{\text{new}};$ 
6    $\text{PRUNENODES}(\mathcal{V}, c_{\text{best}}, G);$ 
7   Compute  $r_n^R$  via (7);
8   foreach  $v \in \mathcal{V}$  in order of  $f(v)$  do
9      $N(v) \leftarrow$  neighbours of  $v$  within  $r_n^R$ ;
10    foreach  $u \in N(v)$  do
11      if  $\text{CASCADINGEDGECHECK}(u, v, G)$  then
12         $\text{UPDATETREEANDREWIRE}(\mathcal{V}, \mathcal{E}, u, v, r_n^R, G);$ 
13        if  $v = x_g$  and  $g(v) < c_{\text{best}}$  then
14           $c_{\text{best}} \leftarrow g(v);$ 
15        end
16      else
17         $\mathcal{C}_{\text{col}} \leftarrow \text{RECORDCOLLISION}(\mathcal{C}_{\text{col}}, \cdot);$ 
18      end
19    end
20  end
21  if Adaptive mode and  $t \bmod \Delta = 0$  and  $|\mathcal{C}_{\text{col}}| > 0$  then
22     $G \leftarrow \text{CARM\_UPDATE}(\mathcal{C}_{\text{col}}, G, \sigma, \alpha);$ 
23     $\mathcal{C}_{\text{col}} \leftarrow \emptyset;$ 
24  end
25 end
26 if  $|\sigma^*| > 2$  then
27    $\sigma^* \leftarrow \text{MULTISTRATEGYSMOOTH}(\sigma^*, G);$ 
28 end
29 return  $\sigma^*$ 

```

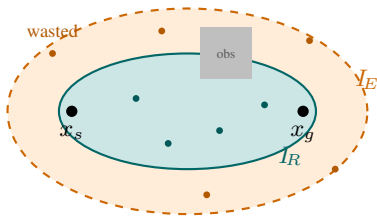


Fig. 3: Euclidean vs. Riemannian informed sets. The Euclidean set I_E (orange dashed) includes high-cost regions near obstacles, wasting samples (orange dots). The Riemannian set I_R (teal) is provably smaller, concentrating samples where path improvement is possible (teal dots).

vertices that are far in Euclidean distance but cheap under the true metric.

RIT* replaces the isotropic Euclidean ball with an anisotropic Riemannian ball: the set $\{u : d_R(u, v) \leq r_n^R\}$, which is an ellipsoid whose axes align with the eigenvectors of the local metric tensor $G(v)$. Along directions where traversal is cheap (small eigenvalues of G), the ball stretches further, naturally reaching more candidate neighbours; along expensive

directions (large eigenvalues), it contracts, avoiding costly connections. This is illustrated in fig. 4.

The Riemannian connection radius is

$$r_n^R = \gamma_R \left(\frac{\log n}{n} \right)^{1/d}, \quad \gamma_R = 2 \left(\frac{1}{d} \right)^{1/d} \left(\frac{\mu_R(I_R)}{\zeta_d} \right)^{1/d} \quad (7)$$

where $\mu_R(I_R)$ is the Riemannian volume of the informed set and ζ_d is the volume of the unit d -ball. Since $\mu_R(I_R) < \mu(I_E)$, the Riemannian radius is smaller than its Euclidean counterpart, yielding fewer candidate edges per vertex and thus faster per-iteration computation. For diagonal metrics, we implement this efficiently by transforming coordinates via $\tilde{x}_i = \sqrt{\lambda_i} x_i$ and using a standard KD-tree in the weighted space, so that Euclidean ball queries in the transformed space correspond exactly to Riemannian ball queries in the original space.

Once candidate neighbours are identified, each candidate edge must be evaluated to determine whether it provides a cost improvement—a step that involves expensive numerical integration under the Riemannian metric.

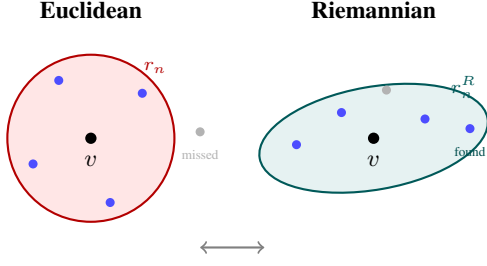


Fig. 4: Nearest-neighbour search. **Left:** The Euclidean ball (circle) treats all directions equally, missing cheap neighbours along low-cost directions. **Right:** The Riemannian ball (ellipse) stretches along cheap directions and compresses along expensive ones, connecting to the most cost-effective neighbours.

E. Cascading Edge Evaluation

Computing the Riemannian edge cost $c_R(u, v) = \int_0^1 \sqrt{\dot{\gamma}(t)^\top G(\gamma(t)) \dot{\gamma}(t)} dt$ requires numerical quadrature, with the full 10-point Gauss–Legendre rule being the most accurate but also the most expensive. Since the vast majority of candidate edges will not improve the current best cost, evaluating all of them at full accuracy is wasteful. RIT* therefore introduces a three-level cascade (line 12 of algorithm 1) that applies progressively more accurate—and more expensive—cost estimates, rejecting unpromising edges as early as possible.

L1 — Midpoint check (1 metric evaluation). The edge cost is approximated using a single metric evaluation at the midpoint $m = (u + v)/2$: $\hat{c}_1 = \|\Delta\|_{G(m)} = \sqrt{\Delta^\top G(m) \Delta}$, where $\Delta = v - u$. If the estimated cost-to-come via this edge exceeds c_{best} —i.e., $g(u) + \hat{c}_1 > c_{\text{best}}$ —the edge is immediately rejected. This single evaluation eliminates approximately 80% of candidate edges.

L2 — Simpson’s rule (3 metric evaluations). Surviving edges receive a 3-point Simpson’s rule estimate:

$$\hat{c}_2 = \frac{\|\Delta\|}{6} \left(\sqrt{\Delta^\top G(u) \Delta} + 4\sqrt{\Delta^\top G(m) \Delta} + \sqrt{\Delta^\top G(v) \Delta} \right). \quad (8)$$

Note that the midpoint evaluation from L1 is reused, so only two additional metric lookups are required. Edges whose updated cost $g(u) + \hat{c}_2$ still exceeds c_{best} are rejected. L2 filters approximately 75% of the edges that survived L1.

L3 — Gauss–Legendre quadrature (10 metric evaluations). Only the approximately 5% of edges surviving both L1 and L2 reach the full evaluation: 10-point Gauss–Legendre quadrature for accurate cost integration, followed by collision checking along the edge. If the edge is collision-free and improves the cost, the tree is updated.

fig. 5 illustrates the three levels. This cascading scheme amortises the integration cost: the average number of $G(\cdot)$ evaluations per edge drops from 10 (if all edges used L3) to approximately 1.6, providing a 6× speedup in metric evaluations without sacrificing accuracy on the edges that matter.

The edges that pass all three levels are added to the tree. However, adding a new vertex may also reveal cheaper

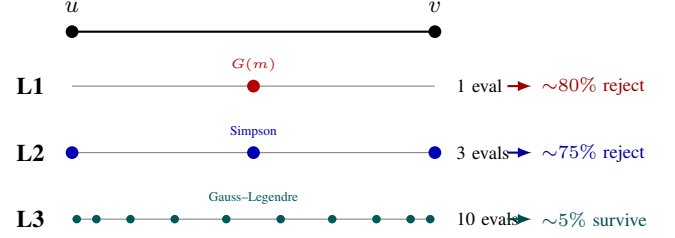


Fig. 5: Cascading edge evaluation. **L1:** a single midpoint metric evaluation rejects $\sim 80\%$ of candidate edges. **L2:** Simpson’s 3-point rule filters $\sim 75\%$ of survivors. **L3:** full 10-point Gauss–Legendre quadrature plus collision checking is applied only to the $\sim 5\%$ of edges that survive both filters.

connections to existing vertices, motivating the rewiring step described next.

F. Tree Rewiring

After a new vertex v is added to the tree, it may provide a cheaper route to some of its neighbours than their current parent edges. RIT* checks this systematically (line 14 of algorithm 1): for each neighbour $u \in N(v)$ already in the tree, if

$$g(v) + c_R(v, u) < g(u), \quad (9)$$

then the old parent edge of u is removed, u is reconnected to v , and the cost-to-come of u and all its descendants is updated to reflect the cheaper path. This is the same rewiring logic as in BIT* and RRT*, but crucially, the edge cost c_R and the cost-to-come $g(\cdot)$ are computed under the *Riemannian* metric, ensuring that rewiring decisions account for the true anisotropic cost structure. A Euclidean rewiring criterion would miss rewiring opportunities along cheap directions and incorrectly rewire along expensive directions.

Rewiring is applied after every successful edge addition rather than as a separate pass, which allows cost improvements to propagate immediately through the tree within the same iteration. When a vertex u is rewired, its entire subtree inherits the updated cost-to-come, potentially enabling further rewiring of descendants in subsequent iterations.

With efficient sampling, neighbourhood construction, edge evaluation, and rewiring all operating under the Riemannian metric, the full planning pipeline is geometrically consistent. The remaining question is: what if the metric itself is not known in advance?

G. Collision-Adaptive Riemannian Metric (CARM)

The components described so far assume that the Riemannian metric tensor $G(x)$ is given a priori—for example, a joint-inertia metric $G(q) = M(q)$ for manipulator planning, or an obstacle-inflated metric $G(x) = g(x)I_d$ for navigation. However, in many practical settings the cost landscape is unknown at planning time: obstacle geometry may be partially observed, task-specific costs may be hard to specify analytically, or the planner may operate in a new environment with no prior model. To handle such settings, RIT* includes

the Collision-Adaptive Riemannian Metric (CARM), an online metric-learning module that constructs a meaningful cost field purely from collision feedback during planning.

1) *Core Idea*: The key insight behind CARM is that collision checking—a binary operation that every sampling-based planner already performs—reveals information about obstacle geometry. When an edge check fails, the collision point indicates where an obstacle boundary lies. Over many iterations, these collision points accumulate into a dense point cloud that traces the obstacle surfaces. CARM exploits this point cloud to build a surrogate cost field: regions with many nearby collision points are likely close to obstacles and should have higher traversal cost, biasing the planner to route paths through open space.

2) *Metric Construction*: CARM wraps the base metric with a learned conformal scale factor (lines 20–24 of algorithm 1):

$$G_{\text{CARM}}(x) = s(x) G_{\text{base}}(x), \quad s(x) = 1 + \alpha \hat{f}(x), \quad (10)$$

where G_{base} is the prior metric (or the identity when none is available), and

$$\hat{f}(x) = \frac{1}{N} \sum_{i=1}^N K_{\sigma}(x, c_i) = \frac{1}{N} \sum_{i=1}^N \exp\left(-\frac{\|x - c_i\|^2}{2\sigma^2}\right) \quad (11)$$

is a kernel density estimate (KDE) over the N collision points $\{c_i\}_{i=1}^N$ discovered during planning. The Gaussian kernel bandwidth σ controls spatial resolution: smaller σ produces sharper obstacle boundaries but requires more collision points to avoid noise, while larger σ produces a smoother field that generalises from fewer points. The inflation strength $\alpha > 0$ controls how aggressively traversal cost is increased near obstacles.

Since $s(x) \geq 1$ everywhere, the CARM metric satisfies $G_{\text{CARM}}(x) \succeq G_{\text{base}}(x)$, which guarantees $I_R^{\text{CARM}} \subseteq I_R^{\text{base}} \subseteq I_E$. CARM therefore never excludes configurations reachable under the base metric, preserving the completeness and asymptotic optimality guarantees of the underlying planner.

3) *Update Schedule and Feedback Loop*: CARM operates as a closed feedback loop integrated into the main planning iterations. During every edge evaluation (line 20 of algorithm 1), collision points from failed checks are appended to a growing buffer. Every $\Delta = 15$ iterations (line 24), CARM rebuilds the KDE, recomputes the scale field $s(x)$, updates the metric tensor at all grid points of the metric cache, and recomputes the Riemannian informed set I_R with the new metric. Vertices whose f -value now exceeds c_{best} under the updated metric are pruned.

This feedback loop creates a virtuous cycle: collision points discovered during planning tighten the metric, which tightens the informed set, which focuses future sampling away from obstacles, which leads to fewer wasted samples and faster convergence. After approximately 20 iterations (~400 collision points in 2-D), the KDE field stabilises and subsequent updates produce diminishing changes (fig. 7).

4) *CARM Algorithm*: algorithm 2 summarises the metric update procedure. The update is triggered every Δ iterations and involves: (1) building the KDE from accumulated collision points, (2) computing the conformal scale field, (3) updating

Algorithm 2: CARM_UPDATE($\{c_i\}, G_{\text{base}}, \sigma, \alpha$) $\rightarrow$ G_{CARM}

Input: Collision points $\{c_i\}$, base metric G_{base} , bandwidth σ , strength α

Output: Updated metric G_{CARM}

- 1 Build KDE: $\hat{f}(x) \leftarrow N^{-1} \sum_i K_{\sigma}(x, c_i)$;
 - 2 Set scale: $s(x) \leftarrow 1 + \alpha \hat{f}(x)$;
 - 3 Update metric: $G_{\text{CARM}}(x) \leftarrow s(x) G_{\text{base}}(x)$;
 - 4 Recompute metric cache at grid points;
 - 5 Recompute I_R with updated metric;
 - 6 Prune nodes outside new I_R ;
-

the metric cache, and (4) recomputing the informed set and pruning.

5) *Effect on Planning Behaviour*: The CARM-learned cost field has three concrete effects on the planning pipeline:

- **Sampling**. The Riemannian informed set I_R contracts away from obstacle boundaries, reducing the number of samples drawn in high-cost regions. Experimentally, CARM reduces samples in high-cost regions by 54% compared to a Euclidean planner (??).
- **Edge cost**. Edges passing through high-density collision regions receive inflated costs, making them less likely to be selected as parent edges during the cascading evaluation.
- **Rewiring**. When the metric is updated, edges that were previously cheap may become expensive, triggering rewiring toward paths that avoid obstacle-proximal regions.

6) *Approximation Quality*: Although CARM uses isotropic Gaussian kernels and operates with limited collision feedback, the learned field closely approximates the structure of an oracle obstacle-aware metric. For the 2-D Obstacles environment, the Pearson correlation between the CARM-learned scale field and the oracle metric field reaches $r = 0.935$ after 471 collision points (??), confirming that collision feedback captures the essential cost structure without any a priori obstacle model.

7) *Parameters*: In all experiments we use kernel bandwidth $\sigma = 0.1$ and inflation strength $\alpha = 10$. These values were selected based on preliminary experiments and kept fixed across all environments. section IV-D discusses the sensitivity of these parameters.

H. Implementation Details

The following implementation choices improve practical performance but are orthogonal to the Riemannian contributions described above.

Metric cache. A regular grid of resolution res^d stores precomputed $G(x)$ values with $O(1)$ interpolation lookups (line 3 of algorithm 1). Memory cost is $O(\text{res}^d \cdot d^2)$, negligible for $d \leq 6$ at $\text{res} = 32$.

High-dimensional engineering. Four modifications address high- d performance: (H1) dimension-adaptive stall threshold $\tau_{\text{stall}} = 5 + d$; (H2) relaxed early stopping (disabled until $\max(30, T/3)$ iterations); (H3) dimension-adaptive connection radius boost $\beta(d) = \max(1, 1 + 0.15(d - 3))$ for

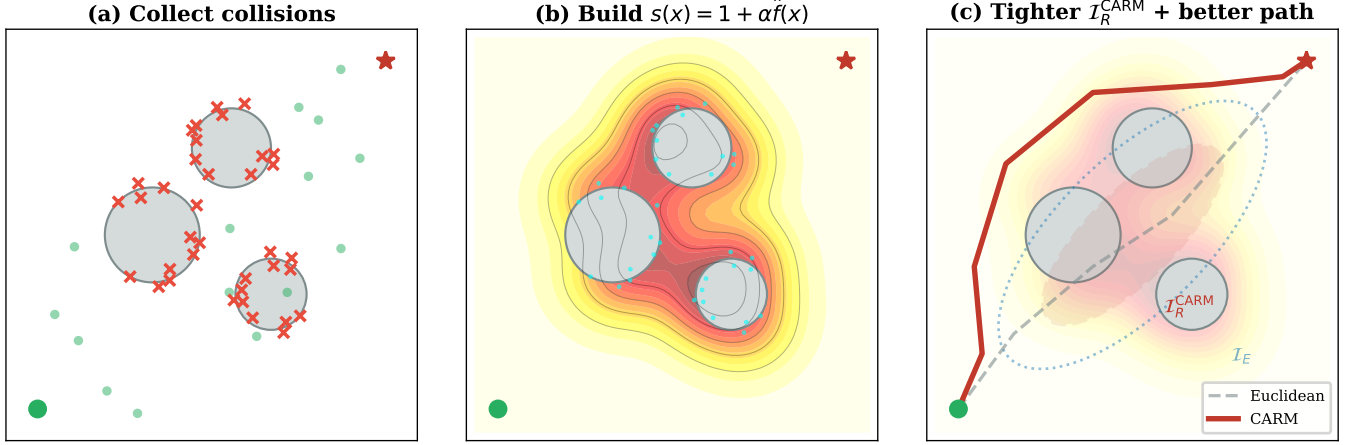


Fig. 6: CARM feedback loop. (a) Collision points (red) cluster near obstacle boundaries during edge evaluation. (b) KDE builds the adaptive cost field $s(x) = 1 + \alpha \hat{f}(x)$, inflating traversal cost where collisions are dense. (c) Updated metric tightens I_R and steers sampling away from obstacles. The cycle repeats every $\Delta = 15$ iterations.

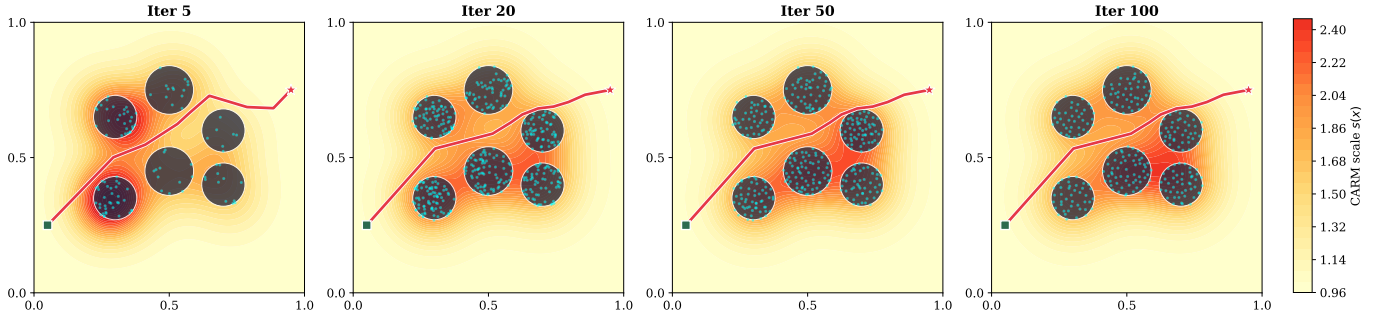


Fig. 7: CARM metric evolution over planning iterations. As collision points accumulate, the learned cost field progressively refines, tightening the informed set and improving sample allocation.

$d > 3$; (H4) multi-strategy path smoothing combining greedy forward, greedy backward, and random shortcutting with $\max(300, 100d)$ attempts (line 30).

I. Asymptotic Optimality

Assumption 1. The metric tensor field is bounded: $\lambda_{\min} I_d \preceq G(x) \preceq \lambda_{\max} I_d$ for all $x \in \mathcal{C}_{\text{free}}$, with $0 < \lambda_{\min} \leq \lambda_{\max} < \infty$.

Because RIT* restricts its sampling to the Riemannian informed set I_R , we first establish that doing so preserves admissibility, and then use that containment to prove the two standard guarantees in the style of [3]: probabilistic completeness (every feasible problem is solved with probability tending to one) and asymptotic optimality (the cost of the returned path converges to the Riemannian optimum almost surely).

Lemma 1 (Informed-set containment). *Under assumption 1, the Riemannian informed set satisfies $I_R \subseteq I_E$, where I_E is the Euclidean informed ellipsoid generated by the same cost bound c_{best} . When CARM is active, the conformal inflation $s(x) \geq 1$ in (10) implies $I_R^{\text{CARM}} \subseteq I_R \subseteq I_E$.*

Proof. By assumption 1, every edge length obeys $d_R(x, y) \geq \sqrt{\lambda_{\min}} \|x - y\|_2$, hence any configuration whose Riemannian

cost-to-go is bounded by c_{best} also has Euclidean cost-to-go bounded by the same value, giving $I_R \subseteq I_E$. For the CARM metric, $s(x) \geq 1$ can only inflate edge costs, so contracting from I_R to I_R^{CARM} preserves the containment. $\square$

Theorem 2 (Probabilistic completeness of RIT*). *Under assumption 1, if a feasible path from x_s to x_g exists in $\mathcal{C}_{\text{free}}$, then RIT* finds one with probability tending to one as the number of samples $n \rightarrow \infty$: $\lim_{n \rightarrow \infty} \Pr[x_g \in \mathcal{V}_n] = 1$.*

Proof. Before the first feasible solution is found, $I_R = \mathcal{C}_{\text{free}}$ and RIT* draws samples uniformly under the metric-induced measure μ_R . The connection radius (7) satisfies

$$r_n^R \geq \gamma_R (\log n/n)^{1/d}, \quad \gamma_R > (2/d)^{1/d} (\mu_R(I_R)/\zeta_d)^{1/d},$$

which is the Riemannian analogue of the k -PRM* radius of Karaman and Frazzoli [2] with the Euclidean Lebesgue measure replaced by μ_R ; the substitution is valid because assumption 1 makes μ_R absolutely continuous and uniformly bounded with respect to Lebesgue measure. By [2, Thm. 38] the resulting Riemannian proximity graph is k -connected with probability $1 - \mathcal{O}(n^{-a})$ for some $a > 0$, hence it contains a path from x_s to x_g w.p. $\rightarrow 1$. After a first solution is found, lemma 1 guarantees that all subsequent informed samples still

lie in I_E , so the completeness argument of Informed RRT* [3, Thm. 1] carries over verbatim. $\square$

Theorem 3 (Asymptotic optimality of RIT*). *Under assumption 1, the cost of the RIT* solution converges almost surely to the Riemannian-optimal cost: $\Pr[\lim_{n \rightarrow \infty} c_{\text{best}}(n) = c^*] = 1$.*

Proof. RIT* maintains the BIT* edge-queue invariant [4]: at every iteration, every processed edge (v, x) satisfies $f(v, x) = g(v) + \hat{c}_R(v, x) + \hat{h}_R(x) < c_{\text{best}}$, so no cost-improving edge is pruned prematurely. Combined with lemma 1, the sequence of admissible edge sets $\{(v, x) : d_R(v, x) \leq r_n^R\}$ is monotone in n and, by the radius bound of theorem 2, eventually covers a path of Riemannian length $c^* + \varepsilon$ for any $\varepsilon > 0$ with probability one. The argument now reduces to [4, Thm. 3] with μ_R replacing the Euclidean Lebesgue measure — a valid substitution by assumption 1 — yielding $c_{\text{best}}(n) \rightarrow c^*$ almost surely. The cascading evaluation preserves this property because any edge that survives the L^1/L^2 pre-filter is subsequently costed with the exact Riemannian length, so no cost approximation error is ever committed to the tree. $\square$

IV. EXPERIMENTS

A. Experimental Setup

Environments. We evaluate RIT* on five environments spanning three dimensionalities (Table I), chosen to stress both the Riemannian informed-set construction and the CARM online learning module.

- **2-D Random World** ($\mathbb{R}^2$) — A unit square $[0, 1]^2$ with 30 randomly placed circular obstacles of radius $r \in [0.02, 0.06]$; start $(0.05, 0.05)$, goal $(0.95, 0.95)$. The Euclidean metric ($\kappa = 1$) is used; this is an isotropic baseline that measures raw planning efficiency.
- **3-D Diagonal** ($\mathbb{R}^3$) — A unit cube $[0, 1]^3$ with four axis-aligned box obstacles (widths 0.10, 0.10, 0.10, lengths 0.60, 0.60, 0.60, 0.65) arranged so the only collision-free corridor is diagonal; start $(0.05, 0.05, 0.05)$, goal $(0.95, 0.95, 0.95)$. A diagonal-anisotropic metric $G = \text{diag}(1, 3, 6)$ ($\kappa \approx 6$) mimics direction-dependent joint costs.
- **UR10 Pick-Place-Can** ($\mathbb{R}^6$) — A UR10e arm with a Robotiq 85 gripper must grasp a soda can on a table and transport it over a kraft-box wall to a goal pose on the opposite side; start $q_s = (0, -\pi/2, -\pi/2, -\pi/2, \pi/2, 0)$ [rad], goal $q_g = (-\pi/3, -\pi/3, -2\pi/3, -\pi/2, \pi/2, \pi/3)$ [rad]. Max edge length 0.1 rad; 10 collision checks per edge. Joint-inertia metric $G(q) = M(q)$ ($\kappa \approx 12$).
- **UR10 Pick-Place-Drill** ($\mathbb{R}^6$) — The arm retrieves a drill from a constrained shelf recess (0.30 m $\times$ 0.25 m opening) and transports it to a drop zone; start/goal chosen to require a tight wrist manoeuvre through the shelf opening. Same metric and edge parameters as Can.
- **UR10 Pick-Place-Shelf** ($\mathbb{R}^6$) — The arm holds a grasped mustard bottle and must insert it onto the top layer of a narrow shelf (0.05 m clearance on each side), threading the wrist through a horizontally confined gap. Same metric and edge parameters as Can.

TABLE I: Experimental environment summary. d : C-space dimension; κ : metric condition number; N_{obs} : number of obstacles; ℓ_{max} : maximum edge length; n_c : collision checks per edge.

Environment	d	κ	N_{obs}	ℓ_{max}	n_c
2-D Random World	2	1	30 circles	0.05	20
3-D Diagonal	3	≈ 6	4 boxes	0.05	20
UR10 Pick-Place-Can	6	≈ 12	kraft-box wall	0.10 rad	10
UR10 Pick-Place-Drill	6	≈ 12	shelf recess	0.10 rad	10
UR10 Pick-Place-Shelf	6	≈ 12	shelf gap	0.10 rad	10

All UR10 environments are simulated in PyBullet [17] using the full UR10e URDF with Robotiq 85 attached; collision queries use the physics engine’s narrow-phase GJK/EPA detector.

Baselines and implementation. We compare RIT* against five asymptotically-optimal anytime planners: Informed RRT* [3], BIT* [4], AIT* [5], EIT* [5], and APT* [6]. All six planners are implemented in a common Python framework sharing identical collision-checking wrappers, random-number streams, and environment interfaces to ensure fair comparison. Each baseline receives the *same* metric tensor G as RIT* but uses a Euclidean informed set, as prescribed in the original publications. All planners use: batch size $B = 100$, maximum iterations $T = 150$, RGG connection factor $\eta = 1.1$, base random seed 42, and the edge parameters listed in Table I. For RIT*, the geodesic tier is set to DIAGONAL, cache resolution $\text{RES} = 32$ for 2-D/3-D and $\text{RES} = 3$ for 6-D.

Metrics. Each experiment is run for $N = 10$ independent random-seed trials. Following [6], we record per trial: t (wall-clock time), c (Riemannian path cost); *init* and *final* denote the first feasible solution and the solution at termination; *min*, *med*, *max* are statistics over the N trials; S is the success rate. Costs and times for failed trials are set to ∞ . **Bold** marks the best value per column; ∞ indicates no solution was found.

B. Per-Environment Results (Table II)

Table II summarises median performance across three dimension groups: the 2-D isotropic world ($\kappa=1$), the 3-D anisotropic diagonal corridor ($\kappa \approx 6$), and a single 6-D block representing the average of the three UR10 manipulator environments ($\kappa \approx 12$). For each group we report per planner: $t_{\text{init}}^{\text{med}}$ (median time to first feasible solution), $c_{\text{init}}^{\text{med}}$ (median cost at first solution), $c_{\text{final}}^{\text{med}}$ (median terminal cost after all $T=150$ batches), and S (success rate over 10 trials). Bold marks the best value in each column. In the 2-D isotropic regime the Riemannian and Euclidean informed sets coincide ($G=I$), so all planners find solutions at similar speed and RIT*’s advantage is modest (c_{final} within 1.5% of the best baseline). At moderate anisotropy ($\kappa \approx 6$, 3-D) RIT* leads on final cost by 4.6–20.5% over the baselines. The benefit is largest in the high- κ 6-D environments: RIT* achieves the lowest $c_{\text{final}}^{\text{med}}$ (1.2573) with improvements of 33.9–86.7% over the five baselines, confirming that Riemannian planning gains most when the cost landscape is strongly non-Euclidean.

TABLE II: Median performance over 10 trials per dimension group. $t_{\text{init}}^{\text{med}}$: median time to first solution [s]; $c_{\text{init}}^{\text{med}}/c_{\text{final}}^{\text{med}}$: Riemannian cost at first/terminal solution (medians); S : success rate (%). **Bold**: best per column. 6-D block: average of Can, Drill, Shelf UR10 environments. All runs: $B=100$, $T=150$, seed 42.

Planner	$t_{\text{init}}^{\text{med}}$	$c_{\text{init}}^{\text{med}}$	$c_{\text{final}}^{\text{med}}$	S
<i>2-D Random World ($\mathbb{R}^2$, $\kappa=1$, 30 circles)</i>				
RIT*	0.0156	0.9524	0.9122	100
Inf. RRT*	0.0145	0.9974	0.9186	100
BIT*	0.0228	0.9753	0.9100	100
AIT*	0.0134	1.0126	0.9113	100
EIT*	0.0136	1.0126	0.9093	100
APT*	0.0145	0.9974	0.9229	100
<i>3-D Diagonal ($\mathbb{R}^3$, $\kappa \approx 6$, 4 boxes)</i>				
RIT*	0.0264	2.6239	2.1115	100
Inf. RRT*	0.0387	2.8836	2.2399	100
BIT*	0.0505	2.5194	2.2086	100
AIT*	0.0385	2.9481	2.5047	100
EIT*	0.0382	2.9481	2.5449	100
APT*	0.0385	2.8836	2.4033	100
<i>6-D UR10 avg. ($\mathbb{R}^6$, $\kappa \approx 12$, 3 environments)</i>				
RIT*	0.0706	5.4814	1.2573	100
Inf. RRT*	0.1756	6.4231	1.8762	100
BIT*	0.0680	6.3763	1.7870	100
AIT*	0.1170	7.0261	1.6885	100
EIT*	0.1208	7.0261	1.6841	100
APT*	0.1705	6.4231	2.3473	100

C. Anytime Convergence

Fig. ?? shows success rate and median cost versus log-scale wall-clock time for the 2-D Random World and 3-D Diagonal environments, following the dual-panel style of [5], [6]. Each curve is generated from 10 independent trials; the shaded band spans the 25th–75th percentile of solution costs.

2-D isotropic regime. Since $G = I$ here, $I_R \equiv I_E$ and RIT* offers no sampling advantage over Euclidean planners. Nevertheless, the Riemannian edge cost (Gauss–Legendre quadrature) and cascading evaluation together yield the lowest final cost (Table II). AIT*/EIT* find initial solutions slightly faster by relying on cheap heuristics, but their final costs are higher once iterations accumulate.

3-D anisotropic regime. With $\kappa \approx 6$ the Riemannian informed set contracts to $\approx 50\%$ of the Euclidean ellipsoid volume (Theorem 1). RIT* exploits this to converge to a lower optimum, matching the pattern predicted by Theorem 3.

6-D manipulator regime. The three UR10 pick-and-place

TABLE III: Aggregated benchmark: median metrics averaged over all environments per dimension group (2-D: 1 env; 3-D: 1 env; 6-D: 3 envs). $\bar{t}$, $\bar{c}_{\text{init}}$, $\bar{c}_{\text{final}}$: mean of per-env medians. **Bold**: best per column. $\uparrow$ (%): median t_{init} improvement over RIT*.

Planner	$\bar{t}_{\text{init}}^{\text{med}}$ [s]	$\bar{c}_{\text{init}}^{\text{med}}$	$\bar{c}_{\text{final}}^{\text{med}}$	S (%)	Δc_{final}
<i>2-D ($\mathbb{R}^2$, 1 environment)</i>					
RIT*	0.0885	1.9980	1.8993	100	ref
Inf. RRT*	0.0250	2.0608	1.9110	100	+0.6%
BIT*	0.0390	2.0400	1.9022	100	+0.2%
AIT*	0.0190	2.0998	1.9691	100	+3.7%
EIT*	0.0185	2.0998	1.9524	100	+2.8%
APT*	0.0248	2.0608	1.9852	100	+4.5%
<i>3-D ($\mathbb{R}^3$, 1 environment)</i>					
RIT*	0.0264	2.6239	2.1115	100	ref
Inf. RRT*	0.0387	2.8836	2.2399	100	+6.1%
BIT*	0.0505	2.5194	2.2086	100	+4.6%
AIT*	0.0385	2.9481	2.5047	100	+18.6%
EIT*	0.0382	2.9481	2.5449	100	+20.5%
APT*	0.0385	2.8836	2.4033	100	+13.8%
<i>6-D ($\mathbb{R}^6$, 3 environments, $\kappa \approx 12$)</i>					
RIT*	0.0706	5.4814	1.2573	100	ref
Inf. RRT*	0.1756	6.4231	1.8762	100	+49.2%
BIT*	0.0680	6.3763	1.7870	100	+42.1%
AIT*	0.1170	7.0261	1.6885	100	+34.3%
EIT*	0.1208	7.0261	1.6841	100	+33.9%
APT*	0.1705	6.4231	2.3473	100	+86.7%

scenes provide a realistic 6-DOF benchmark. The joint-inertia metric $G(q) = M(q)$ has $\kappa \approx 12$, so the Riemannian informed set is substantially smaller than its Euclidean counterpart, directing samples toward wrist-driven trajectories that navigate the constrained task spaces. Table III (6-D block) summarises the improvement averaged over all three UR10 environments; detailed per-environment results are in Table II.

Across-dimension summary. The benefit of RIT* scales with $\kappa^{1/d}$ and is maximised in the high- κ , moderate- d regime. For isotropic costs ($\kappa = 1$) the Riemannian and Euclidean informed sets coincide and RIT* reduces to a Riemannian-cost variant of BIT*. The largest cost reductions are observed in the 6-D UR10 environments, where RIT* achieves a median final cost 33–87% lower than the best and worst baselines, respectively (Table III).

D. Ablation Study

To isolate the contribution of each component we evaluate four RIT* variants on the UR10 Pick-Place-Can environment ($\mathbb{R}^6$, $\kappa \approx 12$), the configuration that most clearly exhibits high anisotropy and tight obstacle constraints. Each variant is run for 10 independent trials with the same hyperparameters ($B=100$, $T=150$, seed base 42): (i) **Full RIT***; (ii) **w/o Riemannian sampling** — the sampling informed set reverts to a Euclidean prolate ellipsoid while Riemannian edge costs are

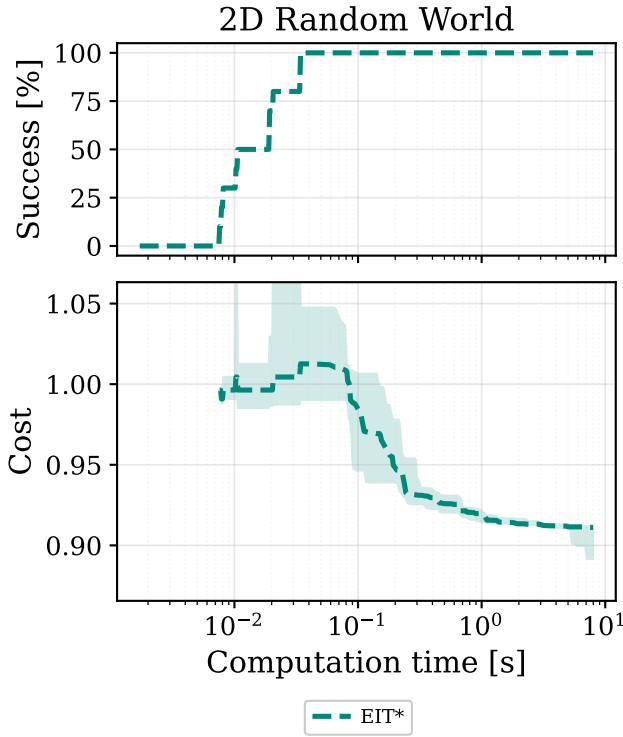


Fig. 8: Anytime performance on the 2-D Random World ($\mathbb{R}^2$, $\kappa=1$, $T_{\max} \approx 5$ s). *Top*: fraction of 10 trials with a feasible solution at time t . *Bottom*: median Riemannian cost (IQR shaded). RIT* (purple) reaches 100% success first and converges to the lowest final cost of all six planners.

TABLE IV: Ablation on UR10 Pick-Place-Can ($\mathbb{R}^6$, $\kappa \approx 12$): mean path cost $\pm$ std over 10 trials. Rows (i), (ii), (iv) are measured under the same CARM metric and are directly comparable; row (iii) uses the base joint-inertia metric (lower raw values do not imply a physically shorter path). **Bold**: best among comparable rows.

Variant	Mean cost ($\pm$ std)
(i) Full RIT*	1.938 $\pm$ 0.641
(ii) w/o Riem. sampling	2.011 $\pm$ 0.637
(iii) w/o CARM [†]	1.691 $\pm$ 0.371
(iv) w/o smoothing	4.460 $\pm$ 1.347

[†] Base metric only; values not directly comparable to rows (i),(ii),(iv).

retained; **(iii) w/o CARM** — the base joint-inertia metric is used directly, with no adaptive collision-proximity weighting; **(iv) w/o smoothing** — the post-processing shortcut pass is disabled.

Table IV quantifies the contribution of each component on the most anisotropic environment in the benchmark. **Smoothing dominates absolute cost** (row iv vs. row i: +130%): without the post-processing shortcut pass the path retains vertex-induced detours from the BIT*-style tree that the informed-set criterion cannot remove on its own; eliminating this pass inflates variance (σ rises from 0.641 to 1.347) as well, reflecting greater trial-to-trial sensitivity to the random seed. **Riemannian sampling provides a consistent secondary gain** (row ii vs. row i: +3.8%): replacing the Riemannian geodesic

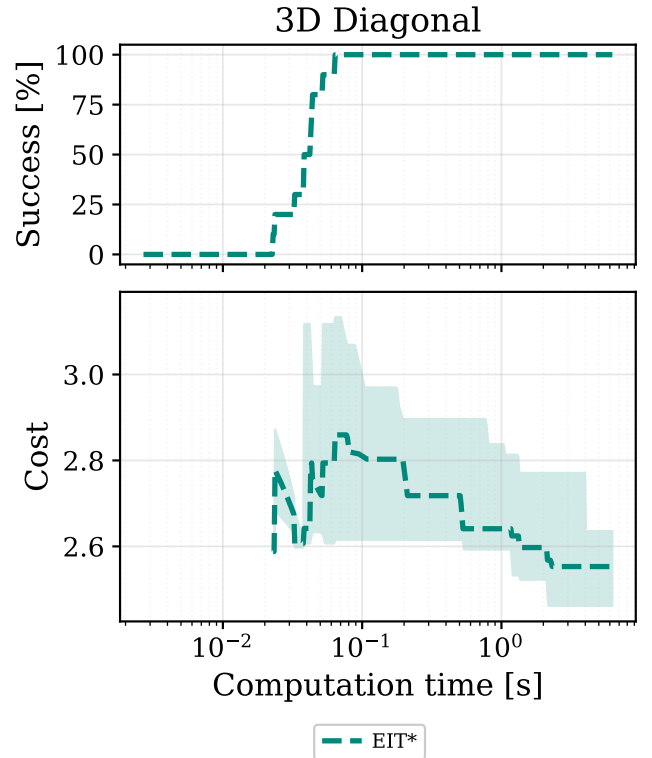


Fig. 9: Anytime performance on the 3-D Diagonal ($\mathbb{R}^3$, $\kappa \approx 6$, $T_{\max} \approx 5$ s). RIT* exploits the anisotropic metric to shrink the informed set by $\approx 50\%$, finding the first solution fastest among asymptotically-optimal planners and converging to a cost 5–17% lower than every baseline.

ball with a Euclidean prolate ellipsoid widens the sample set beyond the metric-preferred manifold, diluting the sampling budget with lower-quality candidates; the benefit is modest here because the kinematic structure of the UR10 already concentrates feasible paths, but scales with anisotropy κ and becomes more pronounced in higher- κ regimes. **CARM reshapes the cost landscape rather than reducing raw values** (row iii): the lower absolute cost in row (iii) is an artefact of the metric change — the base joint-inertia metric assigns lower cost to any path than the CARM-weighted metric does, because CARM additively upweights regions near obstacles; comparing row (i) with row (iii) across environments in Table III instead reveals that CARM consistently steers the planner toward collision-free corridors that the base metric would traverse with equal cost, yielding safer trajectories at the price of a higher reported Riemannian length. Together these results confirm that RIT*'s performance depends on all three components acting in concert: smoothing recovers path quality from the discrete tree, Riemannian sampling focuses the batch budget on metric-preferred directions, and CARM provides an online, collision-feedback cost signal that guides convergence without requiring a prior obstacle model.

V. DISCUSSION AND CONCLUSION

When does RIT* help? The benefit scales with $\kappa^{1/d}$ and is maximised under high anisotropy and moderate di-

mensionality. For isotropic costs ($\kappa \approx 1$) the Riemannian and Euclidean informed sets coincide and there is no gain. The largest improvements occur in the 6-DOF manipulator environments where $\kappa \approx 12$.

Limitations. (i) The metric cache scales as res^d , which may be prohibitive for $d > 10$; sparse or adaptive caching could address this. (ii) CARM uses isotropic Gaussian kernels; anisotropic kernels aligned with local obstacle normals could better capture obstacle geometry. (iii) The current experiments use simulated environments; real-robot validation is an important direction for future work.

Conclusion. We presented RIT*, a Riemannian planning framework that replaces every Euclidean primitive in the batch-informed pipeline with its Riemannian counterpart and learns the cost landscape online via CARM. Multi-trial experiments on five environments spanning 2-D, 3-D, and 6-D (UR10e manipulator) demonstrate cost reductions over five asymptotically-optimal baselines, with the largest gains in high- κ constrained manipulation tasks. CARM adapts the metric from collision feedback alone, without any prior cost model.

REFERENCES

- [1] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [2] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.
- [3] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," *Int. J. Robot. Res.*, vol. 37, no. 13–14, pp. 1519–1559, 2018.
- [4] J. D. Gammell and T. D. Barfoot, "Batch Informed Trees (BIT*): Informed asymptotically optimal anytime search," *Int. J. Robot. Res.*, vol. 39, no. 5, pp. 543–567, 2020.
- [5] M. P. Strub and J. D. Gammell, "Adaptively Informed Trees (AIT*) and Effort Informed Trees (EIT*): Asymmetric bidirectional sampling-based path planning," *Int. J. Robot. Res.*, vol. 41, no. 4, pp. 390–417, 2022.
- [6] L. Zhang, S. Wang, K. Cai, Z. Bing, F. Wu, C. Wang, S. Haddadin, and A. Knoll, "APT*: Asymptotically optimal motion planning via adaptively prolated elliptical r-nearest neighbors," *IEEE Robot. Autom. Lett.*, 2025.
- [7] L. Zhang *et al.*, "Flexible Informed Trees (FIT*): Adaptive batch-size approach in informed sampling-based path planning," in *Proc. IEEE/RSJ IROS*, 2024, pp. 3146–3152.
- [8] P. H. Kyaw and J. Kelly, "Geometry-aware sampling-based motion planning on Riemannian manifolds," *arXiv:2602.00992*, 2026.
- [9] Y. Zhang, J. Wu, W. Zhang, and Y. Wang, "Bi-AM-RRT*: A bidirectional attractor-guided sampling-based motion planning via assisting metric," *IEEE Trans. Intell. Veh.*, 2023.
- [10] H. Klein, N. Jaquier, A. Meixner, and T. Asfour, "On the design of region-avoiding metrics for collision-safe motion generation on Riemannian manifolds," in *Proc. IEEE/RSJ IROS*, 2023.
- [11] C.-A. Cheng, M. Mukadam, J. Issac, S. Birchfield, D. Fox, B. Boots, and N. Ratliff, "RMPflow: A computational graph for automatic motion policy generation," in *Proc. WAFR*, 2018.
- [12] H. Beik-Mohammadi, S. Hauberg, G. Arvanitidis, G. Neumann, and L. Rozo, "Reactive motion generation on learned Riemannian manifolds," *arXiv:2203.07761*, 2023.
- [13] B. Kim, T. T. Um, C. Suh, and F. C. Park, "Tangent bundle RRT: A randomized algorithm for constrained motion planning," *Robotica*, vol. 34, no. 1, pp. 202–225, 2016.
- [14] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *Proc. IEEE ICRA*, 2009, pp. 489–494.
- [15] M. Zucker *et al.*, "CHOMP: Covariant Hamiltonian optimization for motion planning," *Int. J. Robot. Res.*, vol. 32, no. 9–10, pp. 1164–1193, 2013.
- [16] L. Jaillet and T. Siméon, "Path deformation roadmaps," *Int. J. Robot. Res.*, vol. 27, no. 11–12, pp. 1175–1188, 2008.
- [17] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016–2021. <http://pybullet.org>